

Lab report

Implementing and Evaluating KADABRA and BRAVA-GNN in Julia

Martin Schlaier
September 28, 2026

in the Context of the Lab

Computational Analytics

Open-Source Graph Software Engineering

of Prof. Dr. Petra Mutzel
Summer Semester 2026

Supervisor:

Dr. Aurora Rossi
Prof. Dr. Petra Mutzel
Laura Bülte

Fakultät für Informatik
Universität Bonn

Contents

1. Introduction	4
2. Preliminaries	4
2.1. Betweenness Centrality	4
2.1.1. Betweenness Ranking Approximation Problem	5
3. KADABRA	6
3.1. Overview and Problem Formalization	6
3.2. Algorithmic Innovations	6
3.2.1. Balanced Bidirectional BFS (bb-BFS)	6
3.2.2. Rigorous Adaptive Stopping Condition	7
3.3. Theoretical Guarantees	7
3.3.1. Correctness at the Stopping Time	7
3.3.2. Adaptive Top- k Centrality Selection	7
3.3.3. A Disagreement Between the Paper and the Reference Im- plementation	8
3.4. Our Julia Implementation	8
4. BRAVA-GNN	9
4.1. Overview	9
4.2. Key Design Principles	9
4.2.1. Size-Invariant Node Features	9
4.2.2. Dual Message Passing	10
4.2.3. Shortest-Path Preprocessing	10
4.3. Performance	10
5. Experimental Setup	11
5.1. Hardware	11
5.2. Model Training	11
5.3. Instances	11
6. Results	11
6.1. Implementation Comparison: C++ vs. Julia	12
6.2. Thread Scaling	12
6.3. Effect of the Top- k Restriction	14
6.4. BRAVA-GNN vs. KADABRA	15
7. Conclusion	17

A. Supplementary Material	19
A.1. Test Instances	19
A.2. Ground Truth	19
A.3. The Sample Budget ω and Burn-in N_0	20
A.4. KADABRA at the Tight Error Bound	20
A.5. The Top- k Confidence-Budget Allocation	20
A.6. Implementation Capabilities	23
A.7. C++ vs. Julia: Accuracy, and a Normalisation Bug	23
Bibliography	25

1. Introduction

Betweenness centrality measures a node’s importance by the fraction of shortest paths passing through it, and is used to find critical nodes in networks such as power grids, road networks and supply chains. Exact computation relies on Brandes’ algorithm (Brandes 2001), whose $O(|V||E|)$ cost is prohibitive on large graphs, so research has turned to approximation.

Sampling-based methods estimate betweenness from randomly drawn shortest paths with provable error guarantees: RK (Riondato et al. 2014) fixes the number of samples in advance, while ABRA (Riondato et al. 2018) samples progressively until the guarantee holds. **KADABRA** (Borassi et al. 2019) also stops adaptively and makes each sample cheaper with a balanced bidirectional BFS; its authors report average speedups above $100\times$ over previous algorithms on graphs of about 10,000 nodes. It can moreover restrict its guarantee to the top- k nodes.

A second line learns the ranking with Graph Neural Networks. GNN-Bet (Maurya et al. 2019) frames centrality approximation as learning to rank, but its node-specific embeddings make the parameter count grow with the graph; ABCDE (Mirakyan 2021) needs far fewer parameters, using a deep graph convolutional network. **BRAVA-GNN** (Dachille et al. 2026) avoids size-dependent embeddings entirely by using deterministic multi-hop degree features. With about 1,300 parameters, $56\times$ fewer than the lightest prior GNN, it handles directed graphs and is reported to outperform KADABRA in full-graph ranking accuracy. That comparison, however, runs KADABRA only in full-graph mode, although its top- k mode lets it concentrate its effort on the few nodes that matter most.

Our contribution. We contribute (i) a Julia implementation of KADABRA, submitted upstream to `Graphs.jl`, which draws the same number of samples as the reference C++ implementation to within 1.2% and is on average 10% faster (Section 6.1); (ii) a Julia port of BRAVA-GNN with GPU inference; and (iii) a head-to-head accuracy and runtime comparison of the two against exact ground truth, including top- k mode (Sections 6.3 and 6.4). Along the way we identify where the KADABRA reference implementation departs from its paper in top- k mode (Section 3.3.3).

2. Preliminaries

2.1. Betweenness Ranking

The betweenness centrality of a node measures its importance as a bridge or bottleneck based on the fraction of all shortest paths passing through it (Freeman

1977; Brandes 2001). Given an unweighted graph $G = (V, E)$, let $\sigma_{s,t}$ denote the total number of shortest paths from node s to node t , and let $\sigma_{s,t}(v)$ be the number of those paths that pass through $v \in V$. The exact betweenness centrality $bc(v)$ of a vertex v is formally defined as:

$$bc(v) = \sum_{s \neq v \neq t \in V} \frac{\sigma_{s,t}(v)}{\sigma_{s,t}} \quad (1)$$

where $\frac{\sigma_{s,t}(v)}{\sigma_{s,t}} = 0$ if $\sigma_{s,t} = 0$. In literature and practical network libraries, normalized variants are commonly used to ensure scale-invariance across different graph sizes. For instance, the `Julia Graphs.jl` library applies a normalization factor of $\frac{1}{(n-1)(n-2)}$, over all ordered pairs excluding v . Alternatively, the adaptive approximation algorithm KADABRA frames the metric probabilistically as the exact probability that a shortest path between two uniformly selected random nodes passes through v (Borassi et al. 2019). In this convention, the normalization factor is $\frac{1}{n(n-1)}$.

2.1.1. Betweenness Ranking Approximation Problem

Two problems should be distinguished. *Betweenness approximation* asks for estimates $\tilde{bc}(v)$ within a prescribed error of $bc(v)$ for every vertex, with high probability; this is what KADABRA guarantees (Section 3). *Betweenness ranking approximation* asks only that the estimated order of the vertices agree with the true one, which suffices whenever centrality is used to compare vertices, and is what BRAVA-GNN is trained for, since its outputs are scores rather than centrality values (Section 4). The first does not imply the second: vertices whose centralities differ by less than the error bound may be ordered arbitrarily, the effect Section 6.4 measures.

Formally, ranking approximation asks for a size-invariant function $f : V \rightarrow \mathbb{R}$ whose induced order maximizes the rank correlation with the exact centralities:

$$\max_f \text{corr}(\mathcal{R}(f(V)), \mathcal{R}(bc(V))) \quad (2)$$

where $\mathcal{R}(\cdot)$ is the rank operator and corr a rank-correlation index such as Kendall's τ_b . Since only the order matters, the normalization of bc is irrelevant, and the formulation applies unchanged to directed graphs.

3. KADABRA

3.1. Overview and Problem Formalization

KADABRA (**K**ADABRA is an **A**Daptive **A**lgorithm for **B**etweenness via **R**andom **A**pproximation) estimates betweenness by sampling: it draws N shortest paths $\pi_1, \dots, \pi_N$, each by picking two nodes s, t uniformly at random and then a uniformly random st -shortest path, and records an indicator per path,

$$X_i(v) = \begin{cases} 1 & \text{if } v \in \pi_i \\ 0 & \text{otherwise.} \end{cases} \quad (3)$$

Since $\Pr(v \in \pi) = bc(v)$, the sample mean $\tilde{bc}(v) = \frac{1}{N} \sum_i X_i(v)$ is an unbiased estimator of $bc(v)$.

KADABRA minimizes N while ensuring that all estimates lie within an absolute error bound λ with confidence $1 - \delta$ (from Section 3.4 on we write λ as ϵ , matching the `err` parameter of our interface and common literature):

$$\Pr(\forall v \in V, |\tilde{bc}(v) - bc(v)| \leq \lambda) \geq 1 - \delta \quad (4)$$

Rather than fixing N in advance, KADABRA uses **adaptive sampling**: N is a stopping time decided on the fly, capped by a deterministic ceiling ω derived from the graph's estimated diameter (Section A.3).

3.2. Algorithmic Innovations

KADABRA optimizes both the time per sample and the total number of samples required.

3.2.1. Balanced Bidirectional BFS (bb-BFS)

Each sample needs a shortest-path search from s to t . KADABRA replaces the single BFS from s by the bidirectional search standard for Dijkstra's algorithm (Pohl 1971): one BFS from s and one backwards from t , always expanding the frontier of smaller total degree, until the two meet or one runs empty. A connecting edge (v, w) is then chosen with probability proportional to $\sigma_{sv} \sigma_{wt}$, keeping the sampled path uniform among all st -shortest paths. On realistic random graph models this cuts the cost per sample from $\Theta(|E|)$ to $|E|^{\frac{1}{2}+o(1)}$ for degree distributions with finite second moment, and to $|E|^{\frac{4-\beta}{2}+o(1)}$ for power laws with exponent $2 < \beta < 3$.

3.2.2. Rigorous Adaptive Stopping Condition

After each batch of samples, KADABRA evaluates a lower and an upper deviation bound, f and g , for every tracked vertex, and stops once these are tight enough for the task (in absolute-error mode, $f \leq \lambda$ and $g \leq \lambda$ for all vertices). Both bounds depend on per-vertex confidence budgets $\delta_L^{(v)}, \delta_U^{(v)}$, which may be distributed unevenly as long as they sum to $\frac{\delta}{2}$; this freedom later enables the top- k mode.

These budgets must be *fixed before sampling begins*: choosing them from the sample that the stopping condition examines is not covered by the guarantee of Section 3.3.1, a case Borassi et al. (2019) explicitly leave open. The reference implementation therefore draws a fixed burn-in batch, derives the budgets from it, then *discards* those samples and restarts, so that the budgets are independent of the sample they are applied to.

3.3. Theoretical Guarantees

3.3.1. Correctness at the Stopping Time

Because KADABRA decides when to stop from the same samples it reports, the number of samples is itself random, and a bound proven for a fixed sample size does not automatically apply. Borassi et al. (2019) prove that the (λ, δ) -approximation guarantee still holds when the algorithm stops; we rely on this result without reproducing its proof.

3.3.2. Adaptive Top- k Centrality Selection

Where only the most central vertices matter, a uniform λ -approximation of every vertex is wasteful, and can still be too coarse to order them. KADABRA's top- k mode instead guarantees that every vertex either has its exact rank established, is excluded from the top k , or has its estimate within λ .

It sorts the vertices by estimated centrality into $v_1, \dots, v_n$ and stops only once every vertex whose own bounds are still wider than λ satisfies one of:

- **Internal Separation** ($i \leq k$): the interval of v_i is disjoint from those of its rank neighbours v_{i-1} and v_{i+1} .
- **External Exclusion** ($i > k$): $\tilde{bc}(v_k) - f(v_k) \geq \tilde{bc}(v_i) + g(v_i)$, certifying that v_i ranks below v_k .

Because these are *alternatives* to the plain λ criterion, not additional requirements, the top- k condition is easier to satisfy than the absolute-error one *for a fixed*

allocation of the budgets: a vertex whose rank cannot be resolved is simply reported to within λ .

KADABRA also adapts the allocation: `computeDelta` sizes each interval to the rank gaps it must resolve rather than to a uniform λ , so v_i in the top k gets intervals that separate it from v_{i-1} and v_{i+1} , while a vertex outside needs only an upper bound tight enough to fall below v_k . This aims to reach the stopping condition with fewer samples; Section 6.3 tests whether it does.

Where intervals still overlap at the k -th boundary (vertices just outside the top k with upper bounds above v_k 's lower bound), KADABRA returns an extended candidate set of $k' \geq k$ vertices, within which any two are either correctly ranked or almost even ($|bc(v) - bc(w)| \leq 2\lambda$).

3.3.3. A Disagreement Between the Paper and the Reference Implementation

Implementing this allocation exposed a discrepancy. For the first k vertices, Borassi et al. (2019) set

$$\lambda_U(v_i) = \frac{\tilde{bc}(v_{i-1}) - \tilde{bc}(v_i)}{2}, \quad \lambda_L(v_i) = \frac{\tilde{bc}(v_i) - \tilde{bc}(v_{i+1})}{2}. \quad (5)$$

This matches their stopping condition: separating v_i from v_{i+1} requires $\tilde{bc}(v_i) - f(v_i) \geq \tilde{bc}(v_{i+1}) + g(v_{i+1})$, so it is v_i 's *lower* deviation that must clear the gap *below* it. The authors' C++ implementation pairs the two gaps the other way round.

A second divergence concerns the safeguard accompanying Equation 5. When two adjacent estimates are too close to rank, the paper resets all four of their budgets to λ , falling back on the absolute-error criterion for both, and states the rule for adjacent pairs without restricting which. The reference applies it inside the top k and below v_{k+1} , but never to (v_k, v_{k+1}) — the one pair its exclusion test is built around. Their budgets are then not aimed at the λ fallback, and below a gap of 2λ separating them needs bounds tighter than λ : the run oversamples.

Neither divergence affects correctness — the allocation decides *where* the confidence budget is spent, not what is guaranteed — but both cost samples. We follow the paper on both; Appendix A.5 works this through and records two further divergences, all of which NetworKit inherits.

3.4. Our Julia Implementation

The implementation, the benchmark harness and the script that reproduces every measurement in this report are at <https://github.com/Lab-CA-SS26/Lab-CA-SS26-node-importance>. The algorithm is also packaged for

Graphs.jl on the fork <https://github.com/DerSchmachtin/Graphs.jl> (file `src/centrality/kadabra.jl`), from which pull request #518 to JuliaGraphs/Graphs.jl is opened.

It is written against the Graphs.jl `AbstractGraph` interface as `kadabra_centrality`:

```
kadabra_centrality(g, k, err, delta;
                   start_factor = 100, endpoints = false,
                   normalize    = :graphs, parallel  = true,
                   rng           = nothing)
```

returning a named tuple of centralities, lower_bounds, upper_bounds, n_samples, omega and tau; Table 6 in Appendix A.6 summarises what it supports.

`normalize` selects the paper’s $\frac{1}{n(n-1)}$ scaling (`:kadabra`), the $\frac{1}{(n-1)(n-2)}$ of Graphs.jl’s `betweenness_centrality` (`:graphs`), or raw counts (`:none`); the confidence intervals of the stopping condition are returned rather than discarded. Seeded runs are bit-identical only with `parallel=false`: in parallel, thread scheduling decides how many samples each worker contributes before the shared stop.

4. BRAVA-GNN

4.1. Overview

BRAVA-GNN (Betweenness Ranking Approximation via Degree Mass GNN), introduced by Dachille et al. (2026), is a lightweight graph neural network that approximates betweenness rankings on directed and undirected networks. Its main contribution is high ranking accuracy with a parameter count independent of graph size.

4.2. Key Design Principles

4.2.1. Size-Invariant Node Features

Instead of node-specific embeddings, whose size grows with the number of nodes, BRAVA-GNN takes **multi-hop degree mass** as input: the m -th order degree mass aggregates the degrees of all nodes reachable from v by walks of length at most m , weighted by the number of such walks:

$$d^{(m)} = \left(\sum_{k=0}^m A^k \right) d, \quad (6)$$

where A is the adjacency matrix and d the degree vector. This deterministic quantity takes a few sparse matrix–vector products to compute and correlates strongly with betweenness in real networks. Orders 0 through 5 give a fixed six-dimensional input per node, so the parameter count does not depend on graph size.

4.2.2. Dual Message Passing

Shortest paths in directed networks are asymmetric, so BRAVA-GNN runs two message-passing streams: an **incoming** stream over A and an **outgoing** stream over A^T . Both share the same weight matrix at each layer, keeping the architecture compact. Following GNN-Bet (Maurya et al. 2019), a node aggregates only its neighbours’ representations, not its own hidden state.

Each stream is read out at every layer by a shared MLP into an intermediate scalar; summing over layers gives $y_{in}(v)$ and $y_{out}(v)$, which are fused multiplicatively,

$$y(v) = y_{in}(v) \cdot y_{out}(v), \quad (7)$$

which rewards only nodes with strong evidence from *both* directions: a high-betweenness node in a directed graph must be reachable from many sources *and* reach many destinations.

4.2.3. Shortest-Path Preprocessing

Before inference, BRAVA-GNN prunes nodes that provably lie on no shortest path, shrinking the graph the network processes: nodes lacking incoming or outgoing edges, and nodes whose in-neighbours all link directly to their out-neighbours. The paper evaluates these conditions on the undirected graph, ignoring edge orientation; the authors’ code and our port evaluate them on the directed graph, where they are a sound certificate of zero betweenness. On undirected graphs the two coincide.

4.3. Performance

Fixed-dimensional structural features and a weight-shared dual-stream architecture give BRAVA-GNN roughly 1,300 trainable parameters: $56\times$ fewer than the lightest previous GNN and four orders of magnitude below GNN-Bet’s $\approx 58\text{M}$. It nonetheless attains the best Kendall τ_b on all 14 of its test graphs, improving on the strongest baseline by 9.7% on average, with competitive inference time.

5. Experimental Setup

5.1. Hardware

The experiments ran on a server with an AMD Ryzen Threadripper 3960X (24 cores with 2 threads each, 3.80 GHz), 125 GiB of RAM, 768 KiB L1d, 768 KiB L1i, 12 MiB L2 and 128 MiB L3 cache, and two NVIDIA GeForce RTX 2080 Ti GPUs with 11 GiB of VRAM each, under Ubuntu 22.04 LTS. GNN training used a single GPU.

The original C++ KADABRA (Borassi et al. 2019) was compiled with OpenMP, CMake 4.2.1 and GCC 11.4.0; our KADABRA and BRAVA-GNN implementations ran on **Julia 1.12.5**.

5.2. Model Training

To generalise across topologies without domain-specific retraining, BRAVA-GNN is trained entirely on synthetic graphs, in the configuration the original paper’s ablations identify as optimal and generated with the authors’ own scripts: directed Bollobás scale-free graphs (BO), their symmetrized variant (SBO) and uniformly directed hyperbolic random graphs (UDHY). The architecture uses $L = 2$ message-passing layers with hidden dimension $n_{hid} = 12$ and a three-layer prediction MLP ($12 \rightarrow 24 \rightarrow 24 \rightarrow 1$, ReLU, dropout $p = 0.3$), with row-wise L_2 normalization after each message-passing step; training runs for 10 epochs under Adam at a learning rate of 5×10^{-3} .

5.3. Instances

For comparability we use the test instances of Dachille et al. (2026), all from SNAP (Leskovec et al. 2014) and listed in Table 3. Measuring accuracy needs an exact reference ranking: Dachille et al. (2026) provided reference values for five graphs also used by Mirakyan (2021), and we computed exact betweenness for four more (Appendix A.2). Computing it for the remaining five was projected to take days to months each and was not attempted, so all experiments use the nine instances with verified ground truth, shown in bold in Table 3.

6. Results

All benchmarks ran on the hardware of Section 5.1, on SNAP datasets, using the reference C++ KADABRA and our Julia ports.

6.1. Implementation Comparison: C++ vs. Julia

We first compare the Julia port with the reference C++ implementation on the same problem: $\epsilon = 0.0001$, $\delta = 0.1$, $k = 0$ (every vertex to within ϵ , no top- k restriction), a single seed and $-t\ 8$ for both, on 6 graphs spanning three orders of magnitude in size (p2p-Gnutella31, soc-Epinions1, soc-Slashdot0902, email-EuAll, amazon, dblp). Table 1 reports the results.

Graph	C++ time	Julia time	Time ratio	C++ samples	Julia samples	Sample ratio
p2p-Gnutella31	91.78 s	93.62 s	1.02×	51 145 729	51 738 273	1.01×
soc-Epinions1	95.19 s	74.79 s	0.79×	82 899 916	83 060 256	1.00×
soc-Slashdot0902	309.1 s	262.0 s	0.85×	118 156 834	118 342 387	1.00×
email-EuAll	55.85 s	42.14 s	0.75×	54 335 686	54 140 873	1.00×
amazon	7 451.7 s	7 112.5 s	0.95×	109 428 589	109 780 185	1.00×
dblp	8 479.9 s	8 807.6 s	1.04×	44 507 467	44 668 275	1.00×

Table 1: KADABRA C++ vs. Julia at $\epsilon = 0.0001$, $\delta = 0.1$, $k = 0$, single seed, *matched* thread counts ($-t\ 8$ for both). Time ratio mean 0.90×, median 0.90×. Sample ratio (Julia/C++) mean 1.00×, median 1.00×.

Sample counts agree to within about one percent (mean and median ratio 1.00×, range 0.996–1.012×), so both realise the same adaptive confidence calibration (Section 3.2.2). The reference tests the stopping condition every 11 samples per thread; our port, whose threads keep separate counters, only every $\omega/1000$, but halts every thread as soon as a check succeeds. Accuracy is equal too: against the corrected ground truth (Appendix A.2) both score Kendall τ_b between 0.89 and 0.995, within 0.003 of each other on every graph, with a top-100 overlap of 98–100. Appendix A.7 compares them in detail and documents a normalisation bug they shared.

Wall-clock times are close as well, with Julia ahead on four of six graphs (mean and median Julia/C++ ratio 0.90×, range 0.75–1.04×). KADABRA’s inner loop is a bidirectional BFS rather than arithmetic, so both are limited by memory latency rather than by code generation, and a well-typed Julia port has little reason to trail optimised C++. This holds at a tight error bound, where runs take minutes to hours and fixed per-process costs vanish; Section 6.2 shows what happens when they do not.

6.2. Thread Scaling

To check that the $t = 8$ used throughout is a sound choice, we swept $t \in \{1, 2, 4, 8, 16, 24, 32, 48\}$ on the 48-thread machine of Section 5.1 at $\epsilon = 0.01$, $\delta = 0.1$, $k = 0$, with three seeds per configuration, on four graphs spanning two orders of magnitude in size. Figure 1 plots the median with min–max whiskers.

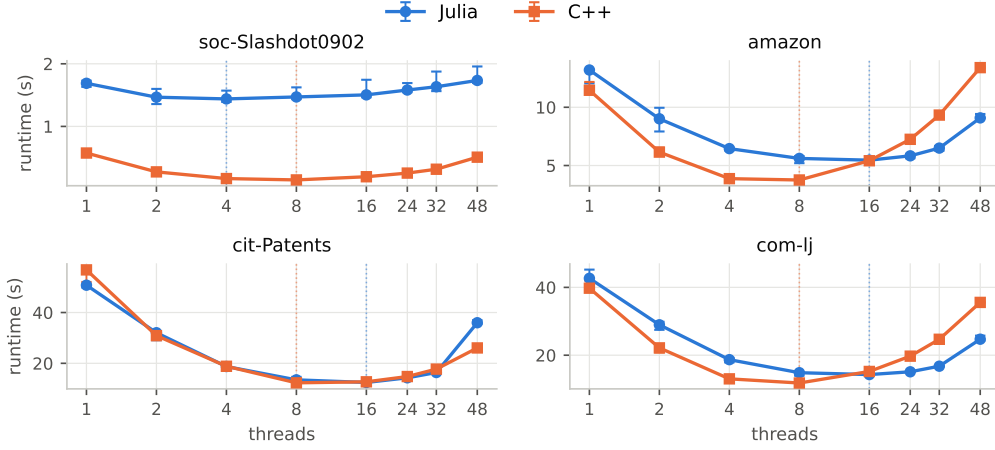


Figure 1: KADABRA runtime against thread count ($\epsilon = 0.01$, $\delta = 0.1$, $k = 0$; median of three seeds, whiskers min-max). Dotted vertical lines mark each implementation’s optimum. Both curves are U-shaped: past the optimum, added threads make the run *slower*.

Optimum well below core count. C++ peaks at $t = 8$ on every graph, Julia at $t = 16$ ($t = 4$ on the nearly flat soc-Slashdot0902 curve), within 9% of its $t = 8$ time; we therefore use $t = 8$ for the matched comparison of Section 6.1.

Degradation at high thread counts. Past the optimum, more threads make runs slower: at $t = 48$ C++ is 2.1–3.6 $\times$ slower than at $t = 8$, Julia 1.2–2.7 $\times$, for different reasons. C++ draws as many samples at $t = 48$ as at $t = 1$ (0.88–1.01 $\times$) but updates shared counters under a global lock after every sample, so its threads contend for that lock and for memory bandwidth. Julia’s threads sample without locking, but a stopping check runs only when a thread completes a batch (1000 samples at this ϵ), so the stop is noticed later the more threads there are: at $t = 48$ Julia draws 1.8–2.9 $\times$ the samples of $t = 1$. Peak speedup is therefore only 3.1–4.6 $\times$ for C++ and 2.4–4.1 $\times$ for Julia (1.2 $\times$ on soc-Slashdot0902), far from linear.

Fixed costs dominate on small graphs. On soc-Slashdot0902 the Julia curve is nearly flat (1.69s at $t = 1$ vs. 1.44s at the optimum): the run lasts under two seconds, mostly scheduler startup. This is why C++ leads by 2.9–10 $\times$ here while the two are close at $\epsilon = 0.0001$ (Section 6.1); at a loose bound the benchmark measures start-up overhead, not sampling throughput.

6.3. Effect of the Top- k Restriction

Restricting KADABRA to the k most central vertices should save computation, yet at $\epsilon = 0.0001$ it cost more on several instances. Tracing this led to the discrepancy of Section 3.3.3. The comparison below switches only the confidence-budget allocation inside our Julia port. To check that the port stands in faithfully for the reference, we also ran the C++ binary in top- k mode on the same four graphs, error bound and seeds; configured to match it, our port draws 1.01 ± 0.12 times its samples, consistent with the $1.00\times$ of Section 6.1 at $k = 0$.

Measured against its own absolute-error runs ($k = 0$), the C++ reference draws $1.77\times$ the samples on email-EuAll at $k = 3$ and $1.76\times$ on soc-Epinions1 at $k = 5$. The penalty lies well outside seed-to-seed variance and appears in 6 of the 16 configurations timed; in our port, scheduling jitter masks it at $k \in \{3, 10, 100\}$ on soc-Epinions1. No guarantee is broken, as every run still meets the requested error bound.

Borassi et al. (2019) could not have seen the effect: they evaluated top- k mode on IMDB and Wikipedia networks through standalone wall-clock times only, and compared against other algorithms in absolute-error mode, whereas the penalty shows only in the ratio of top- k to absolute-error runs.

Both divergences of Section 3.3.3 contribute. Correcting the gap pairing alone makes top- k uniformly cheaper but leaves the same three configurations above $k = 0$ ($1.24\times$, $1.68\times$ and $1.43\times$); only restoring the collapse rule at the boundary removes the effect. Both repairs follow the paper, and both are part of our pull request #518 to `Graphs.jl`.

Figure 2 shows the result, each ratio taken against the same seed’s $k = 0$ run so that shared sampling noise cancels. In none of its twenty configurations across six graphs does the repaired version exceed $k = 0$ by more than one standard deviation (at most $1.01\times$), against 6 of 16 for the reference implementation and 3 of 16 for the reference allocation in our port. On the four graphs run under both allocations it draws $0.69\times$ the samples of $k = 0$, against $0.98\times$ (per graph in Table 5, Appendix A.5), and the top- k answer does not change: the overlap with the exact ranking is identical in all sixteen configurations and the top- k Kendall τ_b agrees to within 0.001.

On `dblp`, the only large instance where the original single-seed sweep put a top- k run above $k = 0$ ($1.06\times$), the repaired allocation gives 0.90 ± 0.06 over three seeds.

Even repaired, the saving varies strongly between graphs: `soc-Slashdot0902` needs only $0.12\times$ the samples at $k = 3$, while `soc-Epinions1` saves nothing at $k = 10$ and $k = 100$ ($1.01\times$ at both). It cannot be predicted from k , because the ceiling ω and the stopping rule depend on ϵ , δ and the estimated diameter but not on k (Appendix A.3); k changes only

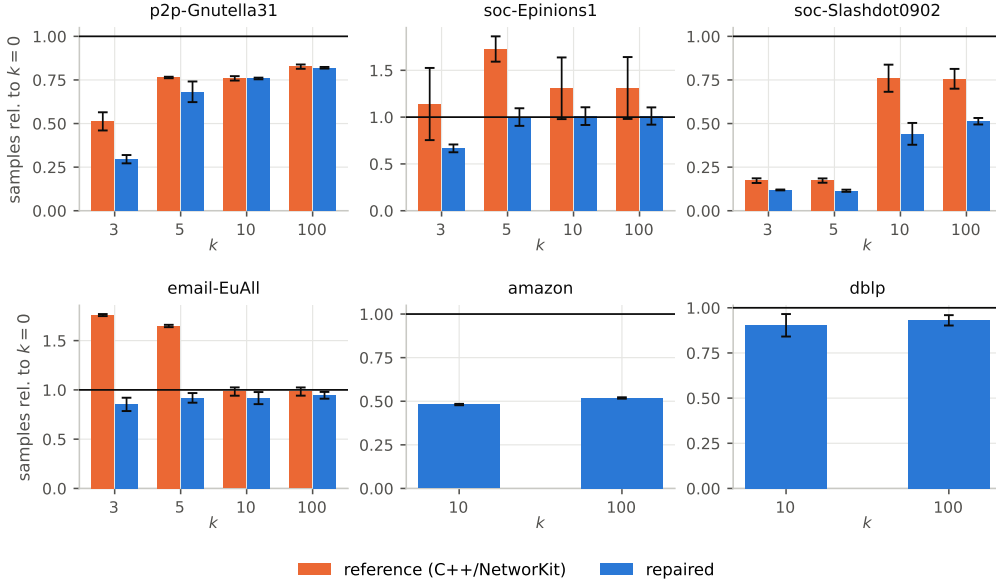


Figure 2: Samples drawn under the top- k restriction relative to $k = 0$ ($\epsilon = 0.0001$, $\delta = 0.1$, $t = 8$; mean over three seeds, whiskers one standard deviation). Both arms are our Julia port with the allocation switched; below the line is a saving. The reference’s allocation rises above the line on `soc-Epinions1` and `email-EuAll`, the repaired one nowhere by more than 1%. `amazon` and `dblp` were too expensive to run under both allocations and carry only the repaired one.

which intervals must be separated, not the budget for separating them.

The grid itself was limited by cost: at $\epsilon = 0.0001$ a run takes minutes to hours and Figure 2 already rests on 126 runs, so `amazon` and `dblp`, at two to three hours per run, carry only the repaired allocation at $k \in \{10, 100\}$.

6.4. BRAVA-GNN vs. KADABRA

We compare KADABRA with BRAVA-GNN, our Julia port of the degree-mass GNN of Section 4, at $k = 0$, $t = 8$ and KADABRA’s standard $\epsilon = 0.01$, on our 9 test graphs. Accuracy is averaged over three repetitions: three sampling runs for KADABRA, and three separately trained models for BRAVA-GNN, whose inference is deterministic given a checkpoint. BRAVA-GNN runs on CPU and on GPU (an NVIDIA RTX 2080 Ti). Table 2 and Figure 3 report the results.

At a loose error bound, the methods exhibit complementary strengths...

At $\epsilon = 0.01$ the two fail in opposite directions. BRAVA-GNN has the higher

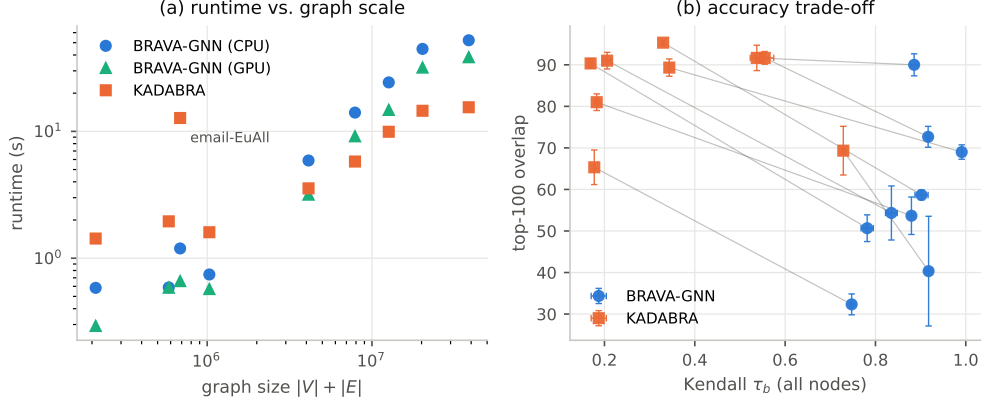


Figure 3: BRAVA-GNN vs. KADABRA at $\epsilon = 0.01$, $\delta = 0.1$, $k = 0$, $t = 8$. (a) Runtime against graph scale (log-log): BRAVA-GNN is faster on the four smallest graphs, KADABRA from $|V| + |E| \approx 4 \times 10^6$ against CPU and from 8×10^6 also against GPU inference; email-EuAll is the exception, small but expensive for KADABRA. (b) The accuracy trade-off, one line per graph joining its two measurements, each a mean over three repetitions with whiskers at one standard deviation. BRAVA-GNN sits to the lower right (better global τ_b , worse top-100 overlap) and KADABRA to the upper left on every graph, though the overlap whiskers meet on soc-Slashdot0902; the τ_b whiskers are at most ± 0.02 . Runtimes in (a) are from the single timed pass.

Kendall τ_b on all nine graphs, often by a wide margin (0.782 vs. 0.169 on com-1j), whereas KADABRA consistently has the better top-100 overlap (90 vs. 51 on com-1j). This follows from KADABRA’s *absolute* error bound: $\epsilon = 0.01$ gives it no way to separate the many vertices with centrality well below ϵ , which make up most of the graph and so dominate τ_b , while its sampling concentrates on the central ones. The τ_b gap exceeds run-to-run variance everywhere, the overlap gap not always: on soc-Slashdot0902 the two lie within one standard deviation (90 ± 3 for BRAVA-GNN against 92 ± 3). BRAVA-GNN’s overlap is also seed-sensitive (Table 2): its three models scored 43, 52 and 26 on p2p-Gnutella31, so its ranking of the very top is a tendency rather than a stable prediction.

...but this dichotomy is largely an artefact of the sample budget. Tightening KADABRA’s bound to $\epsilon = 0.0001$ largely removes the trade-off. On all nine instances KADABRA reaches a τ_b of 0.896–0.995 and a top-100 overlap of 98–100 (per-graph values in Table 4). It wins the overlap on *every* instance and τ_b on eight of nine. The exception is email-EuAll, where BRAVA-GNN keeps the higher τ_b

Graph	BRAVA (CPU)	BRAVA (GPU)	KADABRA	τ_b BRAVA	τ_b KADABRA	Ovlp BRAVA	Ovlp KADABRA
p2p-Gnutella31	0.58 s	0.29 s	1.43 s	0.918 ± 0.006	0.729 ± 0.003	40 ± 13	69 ± 6
soc-Epinions1	0.59 s	0.58 s	1.95 s	0.917 ± 0.004	0.555 ± 0.020	73 ± 3	92 ± 2
soc-Slashdot0902	0.74 s	0.57 s	1.60 s	0.886 ± 0.007	0.537 ± 0.015	90 ± 3	92 ± 3
email-EuAll	1.19 s	0.66 s	12.74 s	0.991 ± 0.000	0.343 ± 0.000	69 ± 2	89 ± 2
com-youtube	5.89 s	3.17 s	3.55 s	0.902 ± 0.015	0.330 ± 0.002	59 ± 1	95 ± 1
amazon	14.06 s	9.14 s	5.78 s	0.835 ± 0.013	0.206 ± 0.009	54 ± 7	91 ± 2
dblp	24.32 s	14.75 s	9.93 s	0.880 ± 0.005	0.183 ± 0.004	54 ± 5	81 ± 2
cit-Patents	44.66 s	31.81 s	14.53 s	0.747 ± 0.005	0.177 ± 0.002	32 ± 3	65 ± 4
com-lj	52.27 s	38.40 s	15.49 s	0.782 ± 0.014	0.169 ± 0.004	51 ± 3	90 ± 1

Table 2: BRAVA-GNN vs. KADABRA ($\epsilon = 0.01$, $\delta = 0.1$, $k = 0$, $-t = 8$) on the 9 test graphs. “ τ_b ” is Kendall τ_b over all nodes; “Ovlp” is top-100 overlap with the exact ranking, out of 100. Accuracy is a mean $\pm$ standard deviation over three repetitions.

(0.991 vs. 0.896): 96% of its vertices have betweenness exactly zero, and BRAVA-GNN’s preprocessing gives this whole block one score, matching the ground truth. Ultimately, what initially appears as complementary strengths is simply a trade-off between accuracy and runtime. With enough samples KADABRA wins on accuracy, at 35–1353 $\times$ the wall-clock time (7968s against 5.89s on com-youtube). BRAVA-GNN’s advantage is not a better ranking of the bulk of the graph, but an accurate global ranking at a speed KADABRA cannot match under tight error bounds.

At $\epsilon = 0.01$, BRAVA-GNN is faster on the four smallest graphs, by up to 10.7 $\times$ on email-EuAll. From com-youtube upward KADABRA overtakes CPU inference, and from amazon upward GPU inference as well (15.5s against 38.4s on com-lj). The two scale differently: at a loose ϵ KADABRA’s sample budget is small and grows slowly with the graph, whereas BRAVA-GNN computes its degree-mass features over the entire graph, so its runtime tracks $|V|$ and $|E|$. Where the crossover lies therefore depends on ϵ and on the hardware. The GPU sped BRAVA-GNN up by 1.4–2.0 $\times$ on seven of nine graphs (on small graphs such as soc-Epinions1, transfer overhead cancels the gain), and its authors report far shorter runtimes on more powerful hardware. The GPU does not change BRAVA-GNN’s accuracy, as the arithmetic is identical on both devices.

7. Conclusion

We implemented KADABRA (Borassi et al. 2019) and BRAVA-GNN (Dachille et al. 2026) in Julia and submitted the former upstream to `Graphs.jl`. We compared our KADABRA port against the authors’ C++ implementation, and both methods against exact betweenness on nine SNAP networks.

Our Julia port is competitive with the optimised C++ reference: at $\epsilon = 0.0001$ the two draw essentially the same number of samples (mean ratio 1.00 $\times$) and

take similar wall-clock time (mean ratio $0.90\times$, i.e. Julia slightly faster). At $\epsilon = 0.01$ C++ is up to $10\times$ faster on small graphs, where sub-second Julia runs are dominated by start-up overhead. Both peak at $t = 8\text{--}16$ threads and slow down beyond.

Implementing top- k mode surfaced two discrepancies between the paper and the reference implementation (Section 3.3.3). The reference sizes the confidence intervals from the rank gaps differently from the paper, contradicting its own stopping test, and it skips the paper’s tie-collapse rule for the one pair its exclusion test depends on. Following the paper on both removes the cases where a top- k query costs *more* than computing every vertex, and brings the sample count down to $0.69\times$ that of an absolute-error run, against $0.98\times$ under the reference’s allocation, with the same top- k answer. Our port includes both corrections, departing from the reference and from NetworkKit, which copies it verbatim.

Which of the two methods is more accurate depends on the error bound. At $\epsilon = 0.01$ BRAVA-GNN gives the better global ranking (higher Kendall τ_b) and KADABRA the better top-100 overlap. At $\epsilon = 0.0001$ KADABRA takes the overlap on every instance and τ_b on eight of nine; the exception is `email-EuAll`, where BRAVA-GNN’s preprocessing captures a large block of zero-betweenness vertices exactly. This accuracy costs $35\text{--}1353\times$ BRAVA-GNN’s runtime, and the top- k restriction does not close that gap: even its largest saving ($0.12\times$ the samples, on `soc-Slashdot0902` at $k = 3$) falls well short of it. At $\epsilon = 0.01$, in contrast, KADABRA outpaces even GPU-accelerated BRAVA-GNN on large graphs (beyond $|V| + |E| \approx 8 \times 10^6$); where this crossover lies depends on ϵ and on the hardware.

The choice therefore depends on the use case: BRAVA-GNN when a globally faithful ranking is needed quickly, KADABRA when accuracy at the top of the ranking matters most and the time is available. One caveat: thread scaling, the top- k sweep and the BRAVA-GNN comparison use three seeds, the other measurements a single one, and BRAVA-GNN’s top-100 overlap is seed-sensitive (it varied by 26 points on `p2p-Gnutella31`). The most pressing open question is the Pareto frontier between the two regimes: how accuracy grows with sampling effort between $\epsilon = 0.01$ and $\epsilon = 0.0001$, which would settle which method wins under a fixed intermediate time budget.

A. Supplementary Material

A.1. Test Instances

Instance	n	m	m/n	Reference
<i>Used in our experiments</i>				
p2p-Gnutella31	62 586	147 892	2.36	calculated
soc-Epinions1	75 879	508 837	6.71	calculated
soc-Slashdot0902	82 168	948 464	11.54	calculated
email-EuAll	265 214	420 045	1.58	calculated
com-youtube	1 134 890	2 987 624	2.63	provided
amazon	2 146 057	5 743 146	2.68	provided
cit-Patents	3 764 117	16 511 741	4.39	provided
com-lj	3 997 962	34 681 189	8.67	provided
dblp	4 000 148	8 649 011	2.16	provided
<i>Not used</i>				
web-Google	875 713	5 105 039	5.83	—
soc-Pokec	1 632 803	30 622 564	18.75	—
wiki-topcats	1 791 489	28 511 807	15.92	—
wiki-Talk	2 394 385	5 021 410	2.10	—
soc-LiveJournal1	4 847 571	68 993 773	14.23	—

Table 3: Instances used in our experiments: n is the number of vertices, m the number of edges. “Reference” records where each instance’s exact betweenness values come from — *provided* by the BRAVA-GNN authors or *computed* exactly. The five instances without one were not evaluated on; the nine in bold are those all reported experiments use.

A.2. Ground Truth

The ground-truth files provided for 8 of the test graphs were labelled inconsistently with the edge lists: each file’s scores match the graph stored with it, but its node IDs do not match ours. We therefore computed exact betweenness ourselves for p2p-Gnutella31, soc-Epinions1, soc-Slashdot0902 and email-EuAll with the single-threaded `betweenness centrality` of `Graphs.jl` (80 and 169 minutes for the last two, against 97.7s and 96.4s reported by Dachille et al. (2026) on 52 cores), and excluded the other four, together with soc-LiveJournal1, which has no ground-truth file, as too expensive to recompute.

A.3. The Sample Budget ω and Burn-in N_0

KADABRA samples shortest-path pairs in two phases. Phase 1 draws a fixed *burn-in* batch of $N_0 = \lfloor \omega / \text{start_factor} \rfloor$ pairs (default `start_factor` = 100) with no stopping check at all. Here $\omega = \frac{1}{2\epsilon^2} (\log_2(D-1) + 1 + \ln \frac{0.5}{\delta})$ is the KADABRA sample-complexity bound as evaluated by the reference implementation, where D is the estimated diameter.¹ Note that ω and N_0 depend only on ϵ , δ , and D — *not* on k . The $\frac{1}{2}$ in front is not cosmetic: ω is sized so that ω samples *alone* bound the error by ϵ with probability $1 - \frac{\delta}{2}$, which is why the per-vertex budgets that the adaptive stopping rule distributes (Section 3.2.2) may only sum to the remaining $\frac{\delta}{2}$ rather than to δ . Phase 2 then keeps sampling, checking the stopping condition periodically, until either that condition is satisfied or the hard ceiling ω is reached.

Across the six graphs of Table 1, the runs stop after 0.13–0.35 ω samples: at $\epsilon = 0.0001$ the adaptive stopping condition terminates well before the worst-case budget.

A.4. KADABRA at the Tight Error Bound

For KADABRA at $\epsilon = 0.0001$, Table 4 gives the per-graph values behind the ranges quoted in Section 6.4. The three largest instances, `com-youtube`, `cit-Patents` and `com-1j`, took roughly 2, 5 and 10 hours on eight threads, which is why they are reported at a single seed.

A.5. The Top- k Confidence-Budget Allocation

Section 3.3.3 gives the two divergences that matter for the sample count: the reference pairs the rank gaps with the opposite sides of each vertex’s confidence interval to Equation 5, and its tie-collapse guard skips the pair (v_k, v_{k+1}) . This appendix records two further divergences and shows how the skipped pair produces the top- k anomaly. The transposed pairing has two consequences:

- For a vertex in the top k , each bound’s error target is set by the gap on the side that bound does not face in the stopping test.
- The top vertex v_1 has no rank neighbour above it, so the paper’s rule gives it only a lower target; the reference gives it only an upper one and leaves $\lambda_L(v_1)$ unconstrained. Since `computeDelta` sets each δ to roughly $e^{-b\lambda^2/\tilde{b}c}$,

¹Borassi et al. (2019) state this bound as $\frac{c}{\lambda^2} (\log_2(VD-2) + 1 + \ln \frac{2}{\delta})$, with VD the vertex diameter; the reference C++ code instantiates $c = 1/2$ and uses $\ln \frac{0.5}{\delta}$, which is what all ω values reported here reflect. Our Julia port reproduces the code’s convention so that the two implementations remain directly comparable.

Graph	τ_b	Ovlp	samples / ω	Time	vs. BRAVA (CPU)
p2p-Gnutella31	0.995	99	0.16	93.6 s	160×
soc-Epinions1	0.969	100	0.21	74.8 s	127×
soc-Slashdot0902	0.975	99	0.35	262.0 s	352×
email-EuAll	0.896	100	0.16	42.1 s	35×
com-youtube	0.955	100	0.72	7 968.0 s	1 353×
amazon	0.924	100	0.31	7 112.5 s	506×
dblp	0.961	98	0.13	8 807.6 s	362×
cit-Patents	0.943	100	0.13	19 290.0 s	432×
com-lj	0.917	100	0.39	37 776.4 s	723×

Table 4: KADABRA (Julia) at $\epsilon = 0.0001$, $\delta = 0.1$, $k = 0$, single seed, $-t \ 8$ — the per-graph values behind the ranges quoted in Section 6.4 and Appendix A.3. “Ovlp” is top-100 overlap with the exact ranking, out of 100. The last column is the wall-clock ratio against BRAVA-GNN on CPU at $\epsilon = 0.01$, taken from the same timed pass as Table 2; BRAVA-GNN’s accuracy is not repeated here, as Table 2 reports it as a mean over three seeds. email-EuAll is the one instance where BRAVA-GNN attains the higher τ_b (0.991 against 0.896).

an unconstrained target drives that δ to its numerical floor and the bound to its maximum — yet $f(v_1)$ is the only one of v_1 ’s bounds that its separation test uses.

The two further divergences:

- For vertices outside the top k the paper sets $\lambda_U(v_i) = \tilde{bc}(v_k) - \lambda_L(v_k) - \tilde{bc}(v_i)$, subtracting the margin by which v_k may itself slip; the reference adds it, making the target looser than the exclusion test can use.
- For $i > k$ the paper certifies v_i as ranking below v_k using $f(v_k)$, the reference using $g(v_k)$. Over the runs of Section 6.3, this changes the sample count by a factor of 0.996 ± 0.023 , i.e. not measurably.

NetworkKit’s `KadabraBetweenness` reproduces all of these choices verbatim, including the skipped pair, and exposes k as a public constructor parameter.

How the skipped pair causes the top- k anomaly. The top- k criterion is a *disjunction* — resolve the rank, or fall back to $f, g \leq \lambda$ — so at a fixed allocation of the confidence budgets it can never stop later than the absolute-error criterion. The excess comes from the allocation. For a vertex outside the top k , `computeDelta`

writes off the lower bound (its target is 1 in the paper and 10 in the code, while betweenness is at most 1), driving δ_L to its numerical floor. That is harmless as long as the vertex is excluded, since exclusion consults only its upper bound. But excluding v_{k+1} rests on the single gap $\tilde{bc}(v_k) - \tilde{bc}(v_{k+1})$ and needs $f(v_k) + g(v_{k+1})$ to fall below it: when the gap is under 2λ , the two bounds must on average become tighter than the λ that the absolute-error criterion asks for.

Meanwhile neither vertex’s budgets are aimed at the fallback. We replayed seed 1 of `soc-Epinions1` at $k = 5$ under the paper’s allocation, with the burn-in estimates held fixed. The gap is 1.43λ , and 3% before the run stops: v_{k+1} ’s written-off lower bound ($\delta_L = 3.3 \times 10^{-10}$) stands at 1.52λ ; v_k ’s upper target was sized by the wider gap above it, leaving $g(v_k) = 1.52\lambda$ as well; and exclusion misses narrowly, with $f(v_k) + g(v_{k+1}) = 1.48\lambda$. The paper’s tie-collapse rule, applied to this pair, resets all four targets to λ , so both vertices can fall back as they would at $k = 0$.

Graph	k	Samples relative to $k = 0$		
		reference	+ gap pairing	+ boundary pair
email-EuAll	3	1.76 ± 0.01	1.24 ± 0.05	0.85 ± 0.07
	5	1.65 ± 0.01	1.68 ± 0.01	0.92 ± 0.05
	10	0.98 ± 0.05	0.91 ± 0.05	0.92 ± 0.06
	100	0.99 ± 0.04	0.94 ± 0.04	0.95 ± 0.03
p2p-Gnutella31	3	0.52 ± 0.05	0.31 ± 0.00	0.30 ± 0.02
	5	0.76 ± 0.01	0.68 ± 0.06	0.68 ± 0.06
	10	0.77 ± 0.00	0.76 ± 0.00	0.76 ± 0.00
	100	0.82 ± 0.01	0.82 ± 0.00	0.82 ± 0.00
soc-Epinions1	3	1.14 ± 0.39	0.65 ± 0.06	0.67 ± 0.04
	5	1.79 ± 0.01	1.43 ± 0.16	1.00 ± 0.09
	10	1.32 ± 0.34	1.01 ± 0.09	1.01 ± 0.09
	100	1.31 ± 0.33	1.01 ± 0.09	1.01 ± 0.09
soc-Slashdot0902	3	0.17 ± 0.01	0.12 ± 0.00	0.12 ± 0.00
	5	0.17 ± 0.01	0.12 ± 0.01	0.11 ± 0.01
	10	0.76 ± 0.07	0.45 ± 0.06	0.44 ± 0.06
	100	0.76 ± 0.06	0.52 ± 0.01	0.51 ± 0.02

Table 5: Samples drawn under the two published confidence-budget allocations and under the repaired one, relative to the same seed’s $k = 0$ run ($\epsilon = 10^{-4}$, $\delta = 0.1$, $t = 8$; mean $\pm$ standard deviation over three seeds). “Reference” is the C++ implementation’s allocation, “gap pairing” the paper’s (Equation 5, with $\lambda_L(v_k)$ subtracted outside the top k), and “boundary pair” additionally applies the paper’s tie-collapse rule to the pair (v_k, v_{k+1}) . Only the last column never exceeds 1 by more than one standard deviation.

A.6. Implementation Capabilities

Capability	Supported	Notes
Directed / undirected graphs	yes	dispatches on <code>is_directed(g)</code>
Weighted graphs	no	a <code>distsmx</code> argument raises <code>ArgumentError</code>
Absolute error ($k = 0$)	yes	$ \tilde{bc}(v) - bc(v) \leq \epsilon$ for all v , w.p. $\geq 1 - \delta$
Top- k ranking ($k > 0$)	yes	also <code>kadabra_top_k</code> , returning the candidate set
Confidence intervals	yes	per-vertex <code>lower_bounds/upper_bounds</code>
Multi-threading	yes	<code>parallel=true</code> , one <code>Threads.@spawn</code> task per thread
Seeded runs	yes	<code>rng=</code> ; exactly reproducible with <code>parallel=false</code>
Normalization	3 modes	<code>:graphs</code> , <code>:kadabra</code> , <code>:none</code>
Endpoint counting	yes	<code>endpoints=true</code> counts path endpoints

Table 6: Capabilities of our `kadabra centrality` implementation.

A.7. C++ vs. Julia: Accuracy, and a Normalisation Bug

The C++ runs of Section 6.1 record no accuracy of their own, so we re-scored their per-node output against the corrected ground truth with exactly the metrics of our Julia runner. Table 7 compares two C++ and four Julia runs per graph. The two rank equally well: Kendall τ_b lies between 0.893 and 0.995 for both, and the top-100 overlap between 98 and 100. Julia’s τ_b is higher on five graphs and level on the sixth, by at most 0.003.

Graph	Kendall τ_b		Top-100 overlap		Max. absolute error / ϵ		
	C++	Julia	C++	Julia	C++ as output	C++ rescaled	Julia, fixed
p2p-Gnutella31	0.995	0.995	99	98–99	4.9	0.18	0.23
soc-Epinions1	0.967	0.969	100	100	7.8	0.23	0.24
soc-Slashdot0902	0.973	0.975	100	99–100	12.3	0.31	0.44
email-EuAll	0.893	0.896	100	100	4.7	0.20	0.24
amazon	0.923	0.924	100	100	11.6	0.20	0.19
dblp	0.960	0.961	100	98–100	3.5	0.15	0.20

Table 7: Accuracy of the C++ reference and the Julia port at $\epsilon = 0.0001$, $\delta = 0.1$, $k = 0$, $-t = 8$, against the corrected ground truth. τ_b is a mean and overlap a range over two C++ and four Julia runs. The last three columns are the largest $|\tilde{bc}(v) - bc(v)|$ over all vertices, in units of ϵ : the reference’s output as it stands, the same output divided by $1 - N_0/N$, and our port after the fix (one run).

Absolute error, however, exposed a bug the two implementations shared. The reference discards the burn-in counts after calibration (`Probabilistic.cpp`, lines 363–368) but then adds the N_0 burn-in pairs back to the divisor (line 405).

Every estimate therefore divides a count over $N - N_0$ pairs by N , and comes out scaled by $1 - N_0/N$ — here 2.9–7.7% too low. A uniform scaling cannot change a ranking, but it breaks the absolute guarantee: the largest scores are off by 3.5–12.3 ϵ , with 40–89 vertices outside ϵ on every graph. Dividing the reference’s own output by $1 - N_0/N$ brings the maximum error down to 0.15–0.31 ϵ , with no vertex outside. Our port had copied the reference’s divisor faithfully; we changed it to the Phase 2 pair count and upstreamed the fix to `Graphs.jl`, after which the port’s maximum error is 0.19–0.44 ϵ and the absolute guarantee of Table 6 holds. `NetworkKit` divides before adding N_0 and is unaffected. Because the fix rescales every estimate uniformly and does not touch the stopping rule, no ranking, sample count or runtime reported elsewhere in this report changes.

Bibliography

- Borassi, Michele and Emanuele Natale (Feb. 20, 2019). “KADABRA is an Adaptive Algorithm for Betweenness via Random Approximation”. In: *ACM J. Exp. Algorithmics* 24.1, 1.2:1–1.2:35. ISSN: 1084-6654. DOI: 10.1145/3284359. URL: <https://dl.acm.org/doi/10.1145/3284359> (visited on 04/13/2026).
- Brandes, Ulrik (June 2001). “A faster algorithm for betweenness centrality”. In: *The Journal of Mathematical Sociology* 25.2, pp. 163–177. ISSN: 0022-250X, 1545-5874. DOI: 10.1080/0022250X.2001.9990249. URL: <http://www.tandfonline.com/doi/abs/10.1080/0022250X.2001.9990249> (visited on 04/13/2026).
- Dachille, Justin, Aurora Rossi, Sunil Kumar Maurya, Frederik Mallmann-Trenn, Xin Liu, Frédéric Giroire, Tsuyoshi Murata, and Emanuele Natale (2026). “Degree-Mass Message Passing for Betweenness Ranking in Directed and Undirected Networks”. In: *Proceedings of the 35th ACM International Conference on Information and Knowledge Management (CIKM '26)*. Rome, Italy: ACM. DOI: 10.1145/3799682.3840622. arXiv: 2602.09716. URL: <https://doi.org/10.1145/3799682.3840622>.
- Freeman, Linton C (1977). “A set of measures of centrality based on betweenness”. In: *Sociometry* 40.1, pp. 35–41. DOI: 10.2307/3033543.
- Leskovec, Jure and Andrej Krevl (June 2014). *SNAP Datasets: Stanford Large Network Dataset Collection*. <http://snap.stanford.edu/data>.
- Maurya, Sunil Kumar, Xin Liu, and Tsuyoshi Murata (Nov. 3, 2019). “Fast Approximations of Betweenness Centrality with Graph Neural Networks”. In: *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*. CIKM '19. New York, NY, USA: Association for Computing Machinery, pp. 2149–2152. ISBN: 978-1-4503-6976-3. DOI: 10.1145/3357384.3358080. URL: <https://dl.acm.org/doi/10.1145/3357384.3358080> (visited on 04/13/2026).
- Mirakyan, Martin (2021). “ABCDE: Approximating Betweenness-Centrality ranking with progressive-DropEdge”. In: *PeerJ. Computer Science* 7, e699. ISSN: 2376-5992. DOI: 10.7717/peerj-cs.699.
- Pohl, Ira (1971). “Bi-directional Search”. In: *Machine Intelligence* 6. Ed. by Bernard Meltzer and Donald Michie. Edinburgh University Press, pp. 127–140.
- Riondato, Matteo and Evgenios M. Kornaropoulos (2014). “Fast approximation of betweenness centrality through sampling”. In: *Proceedings of the 7th ACM International Conference on Web Search and Data Mining*. WSDM '14. ACM, pp. 413–422. DOI: 10.1145/2556195.2556224.

- Riondato, Matteo and Eli Upfal (2018). “ABRA: Approximating Betweenness Centrality in Static and Dynamic Graphs with Rademacher Averages”. In: *ACM Trans. Knowl. Discov. Data* 12.5, 61, 61:1–61:38. DOI: 10.1145/3208351.

Declaration of AI Usage

Name: Martin Schlaier

I declare that I used the following AI tool(s) in the preparation of this laboratory report:

- AI Tool(s): Anthropic Claude, used through the agentic coding tool Claude Code
- Version/Model (if known): Claude Opus 4.8, Claude Opus 5

Description of AI Use

Claude (Claude Code). From August on, I used Claude as an assistant with access to the repositories, the report sources and the benchmark server. I set the questions it worked on and the rules it had to follow: every number comes from a recorded run on the server and is written into the report by a script, and every claim that can be checked mechanically is checked against the raw results. I decided which findings went into the report and how they are presented, and I worked in the feedback from my supervisor. Within that frame, Claude did substantial work:

- **Debugging and analysis.** Tracing the remaining differences between my port and the C++ reference (the ω formula, the calibration phase, the burn-in normalisation, the stopping coordination); comparing the KADABRA paper with the reference and NetworKit, which led to the top-k allocation findings in Section 3.3.3 and Appendix A.5; and finding why BRAVA-GNN fell short of the paper (a PageRank input channel and a clique-mask mismatch), followed by retraining.
- **Code.** Implementing these fixes and the selectable top-k variants, extending the tests of the `Graphs.jl` fork, and preparing the final version of pull request #518.
- **Experiments.** Writing summary, plotting and diagnostic scripts, running the measurements on the server and writing the scripts to generate the tables and figures from them.
- **Writing.** Drafting and revising text, mostly in Sections 3 and 6, the Conclusion and the Appendix; shortening the report towards the page limit; and LaTeX layout. It also helped draft the slides and speaker notes.

Reproducibility. Because much of this work was done with AI assistance, every result in the report can be checked without relying on it. The repository linked in Section 3.4 contains the raw output of every run, the trained BRAVA-GNN checkpoints and the scripts that produced them. A single script, `reproduce_all.sh` in `benchmark/`, re-runs every measurement on the benchmark server and regenerates every table and figure, together with the numbers quoted in the text. Run on the archived data, it reproduces the report’s tables and figures exactly. Only the exact ground truth is taken as an input rather than recomputed.

I reviewed the AI-generated code and text and take full responsibility for the content of this report.